МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВГУ»)

Факультет прикладной математики, информатики и механики

Кафедра математических методов исследования операций

ОТЧЁТ

по дисциплине «**Компьютерная графика**»

Лабораторная работа № 1

Тема: «Реализация HDR-алгоритма»

Выполнил (а) студент (ка):

Хмель Илья Андреевич

Курс 3 Группа 8 (МО ЭВМ)

Преподаватель: доцент, к.т.н. Сырых А. С.

Воронеж, 2025

Содержание

1. Постановка задачи 3

2. Математическое описание алгоритма 4

3. Программное описание алгоритма 13

3.1. Структура программы и представление данных 13

3.2. Реализация выравнивания 13

3.3. Реализация калибровки Debevec и HDR-слияния 14

3.4. Реализация операторов тонального отображения 15

3.5. Основной сценарий работы программы 17

4. Сравнительный анализ 18

4.1. Методология сравнения 18

4.2. Качественное сравнение 21

4.3. Время выполнения и ресурсы 26

Заключение 28

Список используемых источников 31

Приложение 32

1. Постановка задачи

Цель работы – разработка и реализация алгоритма построения HDR-изображения из серии LDR-кадров с разной экспозицией, включающего этапы выравнивания, радиометрической калибровки и тонального отображения.

Основные задачи:

- изучить принципы формирования HDR-изображений из серии экспозиций;
- реализовать алгоритм выравнивания экспозиций методом Median Threshold Bitmap (MTB);
- реализовать метод Дебевека для радиометрической калибровки отклика камеры;
- реализовать операторы тонального отображения (Reinhard, Drago, Mantiuk);
- провести тестирование алгоритма на синтетических и реальных данных и выполнить качественный анализ результатов.

2. Математическое описание алгоритма

Разрабатываемый алгоритм решает классическую задачу формирования HDR-изображения (High Dynamic Range) из серии LDR-кадров (Low Dynamic Range), снятых при разных выдержках. В общем случае ни одна из исходных фотографий не содержит всех деталей сцены: на одних кадрах «выбиты» света, на других – провалены тени. Цель HDR-алгоритма – восстановить оценку реальной яркости сцены в физических единицах (радианс), а затем отобразить её на обычный экран с ограниченным динамическим диапазоном.

Алгоритм состоит из трех основных «модулей»:

- Выравнивание изображений методом MTB (Median Threshold Bitmap);
- Радиометрическая калибровка и слияние методом Дебевека;
- Тональное отображение (tone mapping) для перевода HDR в LDR с помощью операторов.

Пусть имеется серия изображений одной сцены $I_j(x), j = 1, \dots, N$, снятых с разными выдержками t_j. Необходимо восстановить карту освещенности сцены $E(x)$, независимую от параметров съемки.

Цифровая камера преобразует падающий свет в дискретные значения пикселей. При съёмке с выдержкой t_j (в секундах) измеренный сигнал в точке x_i можно записать как:

$$Z_{ij} = f\left(E_i \times t_j\right),$$

где Z_{ij} – измеренное значение яркости пикселя (0…255) в j-ом кадре для i-й точки сцены, E_i – истинная яркость сцены в этой точке, f – нелинейная функция, отклик камеры.

Функция f включает в себя все нелинейности: усиление, гамма-коррекцию, ограничение по насыщению матрицы. Нам нужно восстановить обратную функцию f^{-1}, или, что удобнее, её логарифм.

Введем $g(z) = \ln f^{-1}(z)$ и получим основное уравнение Дебевека:

$$g(Z_{ij}) = \ln E_i + \ln t_j \tag{1}$$

Стоит заметить, что $g(z)$ зависит только от уровня яркости и одинаков для всех кадров. Что до логарифмов, то $\ln E_i$ есть искомая яркость сцены, а $\ln t_j$ – известный логарифм выдержки. Метод Дебевека строит функцию $g(z)$ и одновременно находит оценки $\ln E_i$ для выбранного набора пикселей.

Реальный снимок содержит миллионы пикселей. Использовать все в системе уравнений нецелесообразно, поэтому выбирается ограниченное число точек $i = 1, \dots, P$ (в этой работе выборка по регулярной сетке). Для каждого такого пикселя и каждого кадра $j = 1, \dots, N$ имеем уравнение (1). Дополнительно вводится весовая функция w(z), уменьшающая влияние пикселей вблизи насыщения (0 или 255), где точность измерений хуже. Классическая треугольная весовая функция Дебевека:

$$w(z) = \begin{cases} z - z_{min}, & z \le \dfrac{z_{min} + z_{max}}{2} \\ z_{max} - z, & z > \dfrac{z_{min} + z_{max}}{2} \end{cases} \tag{2}$$

где $z_{min} = 0,\ z_{max} = 255.$

Тогда взвешенное уравнение для пары (i,j) выглядит так:

$$\mathrm{w}(Z_{ij})\big(g(Z_{ij}) + \ln E_i - \ln t_j\big) = 0 \tag{3}$$

Если решать систему (3) напрямую, можно получить слишком «рваную» функцию $g(z)$, чувствительную к шуму. Чтобы обеспечить плавность отклика, добавляется член, штрафующий вторую разность $g(z)$, то есть дискретный аналог второй производной:

$$\lambda \sum_{z=1}^{254} w(z)\big(g(z-1) - 2g(z) + g(z+1)\big)^2 \tag{4}$$

Здесь λ – параметр регуляризации, задающий баланс между точной подгонкой к данным и гладкостью функции $g(z)$. Чем больше λ, тем более гладким будет отклик и менее чувствительным к шуму.

Получается переопределенная система линейных уравнений относительно $g(z)$ и $\ln E_i$. В этой работе она решается методом Гаусса без использования внешних библиотек.

Для конкретного пикселя (в полной карте, не только в выборке) и фиксированного канала пусть значения интенсивности в кадрах $j = 1, \dots, N$ равны Z_j. Тогда оценка лог-яркости строится как взвешенное среднее:

$$\ln E_i = \frac{\sum_{j=1}^{N} w(z_j) \times (g(z_j) - \ln t_j)}{\sum_{j=1}^{N} w(z_j)} \tag{5}$$

И отсюда:

$$\mathrm{E} = \exp(ln\, E) \tag{6}$$

Таким образом, для каждого пикселя и каждого канала получаются неограниченные 0-255 значения, формирующие HDR-изображение: $HDR(x) = (E_R(x), E_G(x), E_B(x))$.

Метод Дебевека предполагает, что значения Z_{ij} в разных кадрах j относятся к одной и той же точке сцены. На практике при ручной съёмке или мелком сдвиге штатива кадры смещаются на несколько пикселей. Неприменение выравнивания приводит к артефактам, вроде смазанных контуров. Поэтому перед HDR-слиянием выполняется глобальное выравнивание (alignment) всех кадров относительно одного опорного, по сдвигу (dx, dy).

Алгоритм MTB (Median Threshold Bitmap) – быстрый и относительно устойчивый к экспозиции способ сопоставления изображений. Суть этого алгоритма в следующем:

1. Для каждого кадра вычисляется яркость:

$$L(x) = \frac{R(x) + G(x) + B(x)}{3} \tag{7}$$

2. Выбирается порог T. В классическом MTB это медиана яркости на кадре, здесь же – последовательность порогов по битам (например, $T = 2^{7-bit}$) для многомасштабного уточнения.
3. Строится бинарная карта:

$$MTB(x) = \begin{cases} 1, & L(x) \geq T \\ 0, & L(x) < T \end{cases} \tag{8}$$

Пусть имеется эталонное изображение ref и текущее изображение cur. На уменьшенной копии изображений для выбранного порога T вычисляются соответствующие MTB-карты. Для заданного сдвига (dx, dy) оценивается суммарное различие:

$$SAD(dx, dy) = \sum_{x,y} \left| MTB_{ref}(x + dx, y + dy) - MTB_{cur}(x, y) \right|, \tag{9}$$

где суммирование выполняется по тем пикселям, для которых сдвинутые координаты лежат внутри изображения. Это аналог суммы модулей разности (Sum of Absolute Differences) для бинарных изображений.

Далее среди всех сдвигов в окрестности (например, $dx, dy \in [-4,4]$) выбирается тот, для которого SAD минимально. Этот сдвиг считается лучшим приближением к смещению изображений на данном уровне.

Чтобы улучшить качество и устойчивость, применяется такой алгоритм действий:

- Изображение уменьшается (downscale) до фиксированной ширины (например, 512 пикселей) с сохранением пропорций;
- Ищется сдвиг (dx_0, dy_0) по формуле (9);
- Изображение сдвигается, и на новом уровне тестируются малые изменения вокруг (dx_0, dy_0);
- Найденные при каждом шаге приращения суммируются, и получается итоговый сдвиг.

Такая стратегия позволяет быстро найти глобальный минимум на грубом уровне, а затем уточнить его на более детальном, избегая перебора большого числа сдвигов на полном разрешении. Полученный оптимальный сдвиг масштабируется к исходному разрешению. А затем реальное RGB-изображение сдвигается с заполнением пустых областей сплошным черным цветом. Это делается для всех кадров, кроме референсного: так обеспечивается согласованное выравнивание серии.

HDR-карта, полученная по методу Дебевека, содержит значения радианса в расширенном диапазоне и не может быть напрямую отображена на стандартный монитор, который оперирует 8-битными значениями 0–255. Поэтому необходимо выполнить тональное отображение (tone mapping) – сжатие динамического диапазона с сохранением локального контраста и визуальной правдоподобности.

В работе реализованы три оператора тонального отображения: 1) Reinhard (глобальный), 2) Drago (логарифмический, перцептивно ориентированный) и упрощённый 3) Mantiuk (нормировка лог-яркости с контролем контраста).

1) Оператор Reinhard имитирует фотографический процесс экспозиции и проявки, отображая сцены большого динамического диапазона в диапазон, пригодный для печати или отображения.

Вычисление яркости:

$$L_w(x) = \frac{R(x) + G(x) + B(x)}{3} \tag{10}$$

Логарифмическая средняя яркость:

$$L_w = \exp\left(\frac{1}{N}\sum_x ln\big(o + L_w(x)\big)\right), \tag{11}$$

где N – число пикселей, o – константа для избежания записи ln 0.

Нормировка с помощью «ключа» (exposure key):

$$L_m(x) = \frac{k}{L_w} L_w(x), \tag{12}$$

Сжатие динамического диапазона:

$$L_d(x) = \frac{L_m(x)}{1 + L_m(x)} \tag{13}$$

Эта формула отображения имеет асимптоты: при малых L_m яркость почти не изменяется, при больших – приближается к 1, что предотвращает выбивание в чистый белый.

Восстановление цветовых каналов:

$$scale(x) = \frac{L_d(x)}{L_w(x)} \quad (14)$$

$$R_d(x) = R(x) \times scale(x), G_d(x) = G(x) \times scale(x), B_d(x) = B(x) \times scale(x) \quad (15)$$

Таким образом, изменяется только яркость, а относительные соотношения между каналами (цветовой тон) сохраняются.

И, наконец, гамма-коррекция:

$$R_{out} = 255 \times (\max(0, \min(1, R_d)))^{\frac{1}{\gamma}} \quad (16)$$

Аналогично для G_{out} и B_{out}, где $\gamma \approx 2.2$ – типичное значение гаммы для монитора.

Оператор Reinhard является глобальным, т.к. одна и та же функция отображения применяется ко всем пикселям с использованием глобальных характеристик (лог-средней яркости). Подробнее с этим можно будет ознакомиться в программной реализации.

2) Оператор Drago относится к классу перцептуальных операторов, учитывающих логарифмическую чувствительность человеческого зрения. В простейшем виде он использует логарифмическое сжатие яркости с учётом максимальной яркости сцены L_{max} и параметра «bias», задающего предпочтительный диапазон.

Максимальная яркость:

$$L_{max} = \max_x L_w(x) \quad (17)$$

Логарифмическое сжатие:

$$L_d(x) = \frac{\log\left(1 + L_w(x)\right)^{\beta}}{\log(1 + L_{max})}, \quad (18)$$

где β $(0 < \beta < 1)$ – параметр bias: при меньших значениях большее внимание уделяется теням, при больших – светлым участкам.

Восстановление цветовых каналов и гамма-коррекция те же, что и у оператора Reinhard.

Преимущество оператора Drago – более мягкое сжатие ярких областей и чёткая проработка деталей в тенях, что визуально часто даёт более «плоский», но информативный результат.

3) Оператор Mantiuk в полной формулировке основан на более сложной модели восприятия контраста. В данной работе реализован упрощённый вариант, который:

- переводит яркость в логарифмическое пространство;
- нормирует ее по среднему и стандартному отклонению;
- масштабирует контраст;
- возвращается в линейное пространство.

Яркость:

$$l(x) = \ln(L_w(x) + o) \tag{19}$$

Оценка среднего и стандартного отклонения:

$$\mu = \frac{1}{N}\sum_x l(x), \;\; \sigma = \sqrt{\frac{1}{N}\sum_x (l(x) - \mu)^2} \tag{20}$$

Нормировка и масштабирование контраста:

$$z(x) = \frac{l(x) - \mu}{\sigma} \tag{21}$$

$$z'(x) = s \times z(x), \tag{22}$$

где $s \approx 0.7$ – параметр масштаба контраста.

Возврат к линейной яркости:

$$L_d(x) = \exp\big(z'(x)\big) \tag{23}$$

Сохранение цветности с контролем насыщенности:

$$R_n(x) = \left(\frac{R(x)}{R_w(x) + o}\right)^{sat}, G_n(x) = \left(\frac{G(x)}{G_w(x) + o}\right)^{sat}, B_n(x) = \left(\frac{B(x)}{B_w(x) + o}\right)^{sat} \tag{24}$$

$$R_d(x) = L_d(x) \times R_n(x), G_d(x) = L_d(x) \times G_n(x), B_d(x) = L_d(x) \times B_n(x), \tag{25}$$

где параметр sat регулирует насыщенность цветов: при значении <1 картинка становится более блеклой, а при >1 – цвета усиливаются.

Гамма-коррекция и перевод к 0-255 – как в операторе Reinhard.

Преимущество такого подхода – более тонкое управление глобальным контрастом сцены за счет работы в логарифмическом пространстве, а также возможность частичного контроля насыщенности. А в совокупности все эти подсистемы обеспечивают восстановление осмысленной физически HDR-карты освещенности, устраняют смещения между кадрами и переводят HDR-изображение в стандартный 8-битный формат.

3. Программное описание алгоритма

Программная реализация следует математической модели из раздела 2 и организована как последовательный конвейер: загрузка кадров, выравнивание (MTB), калибровка по Дебевеку и построение HDR, тональное отображение тремя операторами, постобработка и сохранение результатов. Реализация выполнена на Python. Для ускорения вычислений используется библиотека NumPy (векторные операции и решение МНК в np.linalg.lstsq), а Pillow применяется для чтения/записи изображений и для части постобработки (ImageEnhance). Специализированные HDR-библиотеки (OpenCV HDR и др.) не используются.

3.1. Структура программы и представление данных

Исходные изображения загружаются функцией read_image_rgb(path), которая открывает файл через Pillow, преобразует в RGB и возвращает массив NumPy типа uint8 размера (H, W, 3) со значениями 0-255.

HDR-изображение (радианс-карта) хранится как массив NumPy типа float32 размера (H, W, 3) с неограниченными сверху значениями (после восстановления радианса). Результаты тонального отображения хранятся как uint8 (H, W, 3).

Программа состоит из следующих основных логических блоков:

- Ввод/вывод изображений: read_image_rgb,save_image_rgb;
- Выравнивание экспозиций MTB по глобальному сдвигу: rgb_to_gray_u8, build_pyramid, mtb_and_exclusion, mtb_error,align_ mtb_translation, а также применение сдвига shift_image_zero_pad;
- Обрезка общей валидной области после выравнивания: compute_common_crop;
- Радиометрическая калибровка Дебевека по каждому каналу: выборка точек pick_sample_points, решение МНК solve_debevec_response;

- HDR-слияние:weight_triangle,merge_hdr_debevec;
- Тональное отображение: tonemap_reinhard, tonemap_drago_simple, tonemap_mantiuk_simple;
- Финальная постобработка: levels_stretch_u8, enhance_pil_pop;
- Основной сценарий запуска: main() и run_pipeline(...) с параметрами через argparse.

3.2. Реализация выравнивания

Блок выравнивания реализован функцией align_mtb_translation(images_rgb, ref_index, pyramid_min_size, tolerance), которая оценивает глобальный сдвиг (dx, dy) каждого кадра относительно опорного.

Основные вспомогательные функции:

- rgb_to_gray_u8: перевод RGB в grayscale для работы MTB только по яркости;
- build_pyramidiresize_gray: построение пирамиды изображений (coarse-to-fine) для ускорения поиска сдвига;
- mtb_and_exclusion: построение MTB-бинарной карты по медиане и exclusion mask (исключение пикселей около медианы, где бинаризация нестабильна);
- mtb_error: вычисление ошибки выравнивания как числа несовпадающих битов MTB с учётом маски исключения;
- shift_image_zero_pad: применение найденного сдвига к полноразмерному кадру с заполнением пустых областей нулями.

Поиск сдвига выполняется пирамидально: на каждом уровне уточняется (dx, dy) локальным перебором 3×3 (±1 пиксель), после чего на следующем уровне сдвиг масштабируется в 2 раза.

Результат работы: align_mtb_translation – список выровненных изображений и список сдвигов shifts.

3.3. Реализация калибровки Debevec и HDR-слияния

Радиометрическая калибровка по Дебевеку реализована функционально и выполняется отдельно для каналов R, G и B.

Функция pick_sample_points(mid_img_u8, n_samples, valid_range) выбирает случайные точки на опорном кадре внутри диапазона интенсивностей (по умолчанию 10…245), чтобы избегать пересветов и глубоких теней.

Для выбранных точек формируется матрица наблюдений Z_samples размера (P, N) (P – число точек, N – число экспозиций).

Функция solve_debevec_response(Z_samples, ln_t, lambda_reg) строит систему уравнений Дебевека с треугольной весовой функцией и регуляризацией гладкости и решает её методом наименьших квадратов через np.linalg.lstsq. На выходе получается дискретная функция отклика g(z)для z = 0…255.

HDR-слияние реализовано функцией merge_hdr_debevec(images_u8, times, g_channels), где images_u8 – стек выровненных кадров размерности (N, H, W, 3). Для каждого канала используется найденная функция отклика g(z) и веса weight_triangle(z). Векторно накапливаются числитель и знаменатель формулы Дебевека для lnE, после чего выполняется E = exp(lnE).

Результат – HDR-радианс карта hdr типа float32 размера (H, W, 3).

3.4. Реализация операторов тонального отображения

Операторы тонального отображения реализованы в виде функций, принимающих HDR-массив float32 (H, W, 3) и возвращающих LDR-изображение uint8 (H, W, 3):

- tonemap_reinhard(hdr, key, gamma) – глобальный оператор Reinhard: вычисляется яркость L, лог-средняя яркость, выполняется сжатие Ld = Lm/(1+Lm), затем восстанавливаются RGB-каналы через масштаб Ld/L и применяется гамма-коррекция. Дополнительно применяется автоматическая

нормировка яркости auto_expose_ldr по перцентилю (подстройка “экспозиции” результата).

- tonemap_drago_simple(hdr, bias, gamma) – логарифмическое сжатие яркости с параметром bias, затем восстановление каналов и гамма-коррекция.
- tonemap_mantiuk_simple(hdr, scale, saturation, gamma) – упрощённый вариант Mantiuk: работа в лог-пространстве яркости, нормировка по среднему и σ, масштабирование контраста и частичный контроль насыщенности, затем нормировка по перцентилю и гамма-коррекция.

3.5. Основной сценарий работы программы

Основной сценарий работы реализован в функциях main() и run_pipeline(...). В main() через argparse задаются: папка входных кадров (--input), выдержки (--times), папка результатов (--output), параметры калибровки (--samples, --lambda_reg) и параметры MTB (--mtb_tol, --mtb_min). Далее run_pipeline(...) выполняет конвейер:

- Загружает изображения read_image_rgb, проверяет одинаковый размер кадров.
- Выбирает опорный кадр как средний по списку (ref_index = N//2).
- Выполняет MTB-выравнивание align_mtb_translation, печатает найденные сдвиги.
- Вычисляет общую область пересечения compute_common_crop и обрезает все кадры до единого размера (уменьшение артефактов и ускорение дальнейших шагов).
- Выполняет калибровку Дебевека по каналам: pick_sample_points solve_debevec_response.
- Строит HDR-карту merge_hdr_debevec.
- Выполняет тональное отображение тремя операторами и сохраняет результаты: 03_tonemap_reinhard.jpg, 04_tonemap_drago.jpg, 05_tonemap_mantiuk.jpg.
- Формирует финальную улучшенную версию: levels-stretch (levels_stretch_u8) + усиление цвета/контраста/резкости (enhance_pil_pop) и сохраняет 06_FINAL_enhanced.jpg.

4. Сравнительный анализ

Задача данного раздела – оценить качество реализованного HDR-конвейера и влияние выбора оператора тонального отображения на итоговое изображение. Сравнение выполняется внутри собственной реализации (без специализированных HDR-библиотек), поэтому основной акцент делается на:

- сравнении исходных LDR-кадров с результатом HDR + тонмаппинга;
- сравнении трёх операторов тонального отображения (Reinhard, Drago, Mantiuk);
- оценке вычислительной трудоёмкости и практической скорости работы конвейера после оптимизаций (пирамидальный MTB, обрезка общей области, векторизация NumPy).

4.1. Методология сравнения

Входные данные:

- серия из 5 изображений (img_0.JPG…img_4.JPG) разрешением 2268 на 4032 пикселей;
- кадры сняты с разными выдержками (−2 EV, −0.7 EV, 0 EV, +1 EV, +1.7 EV; выдержки задаются параметром командной строки --times (в секундах, через запятую). В проведённом эксперименте использовались выдержки: 1/50, 1/33, 1/25, 1/20, 1/17 (вводятся в --times в виде десятичных значений);
- сцена с выраженными контрастами и сложными деталями (рис. 1): яркие области (лампа, свет от мониторов, текстолит видеокарты) и тёмные (фон, тени, вещи на столе).

Рис. 1. Оригинальное тестовое изображение.

Алгоритм обработки:

- Выравнивание MTB;
- Калибровка по Дебевеку и слияние;
- Тональное отображение с помощью операторов.

В процессе работы программа сохраняет промежуточные результаты:

- 01_original_*.jpg – исходные кадры;
- 02_aligned_*.jpg – выровненные кадры;

- 03_tonemap_reinhard.jpg, 04_tonemap_drago.jpg, 05_tonemap_mantiuk.jpg – результаты тонального отображения;
- 06_FINAL_enhanced.jpg – финальный вариант после дополнительной постобработки (levels-stretch + усиление цвета/контраста/резкости).

Критерии оценки результатов:

- визуальная сохранность деталей на свету и в тенях;
- наличие/отсутствие пересветов и провалов в тени;
- естественность цвета и контраста;
- наличие артефактов выравнивания (двоение контуров) и артефактов тонмаппинга;
- скорость работы основных этапов. Отдельно учитывается влияние обрезки общей области (crop) после выравнивания на качество и время выполнения.

4.2. Качественное сравнение

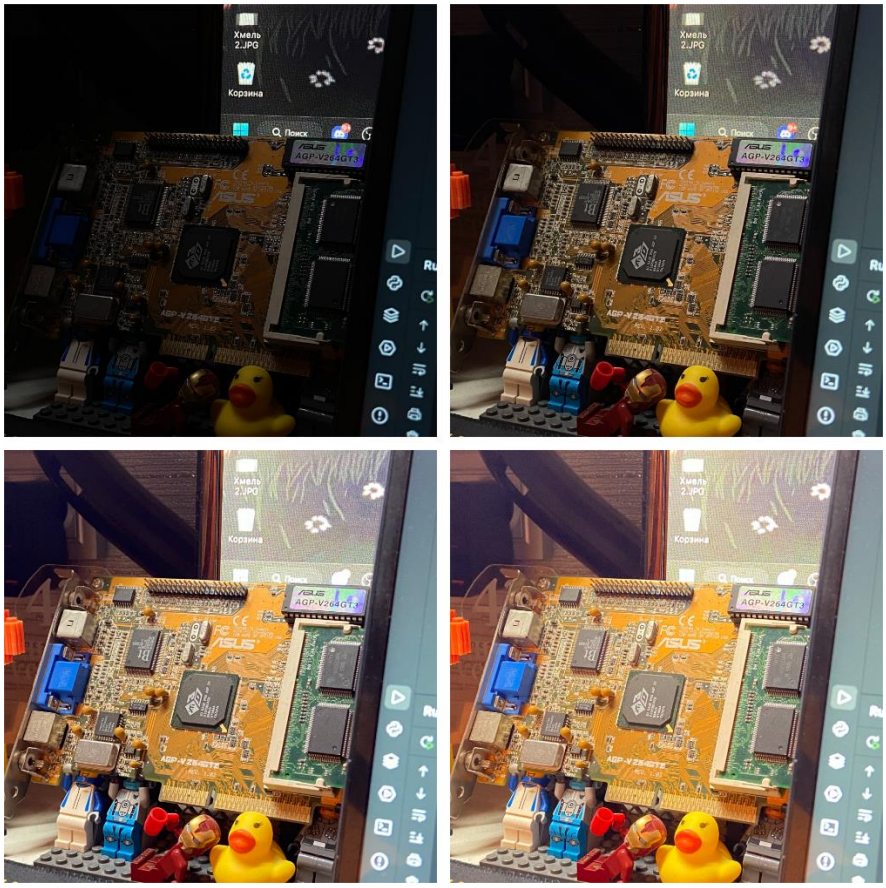

Рис. 2. Серия изображений с разными выдержками.

Рис. 3. Результат HDR + Reinhard.

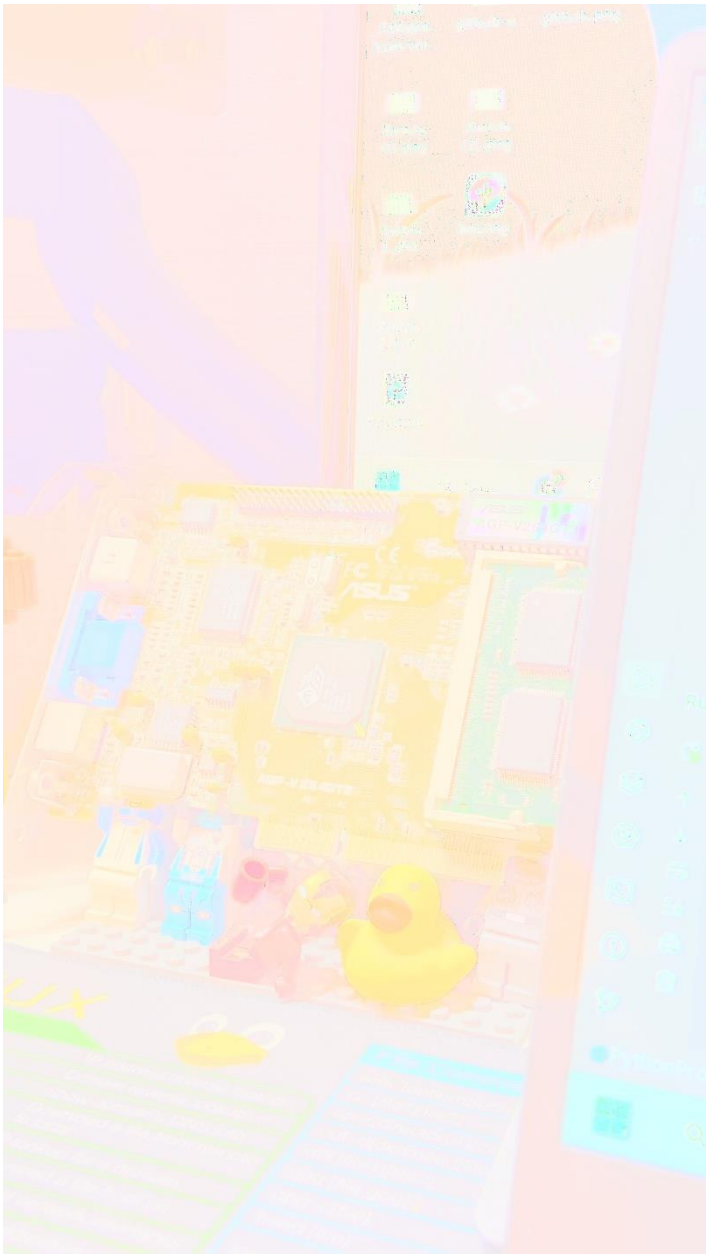

Рис. 4. Результат HDR + Drago.

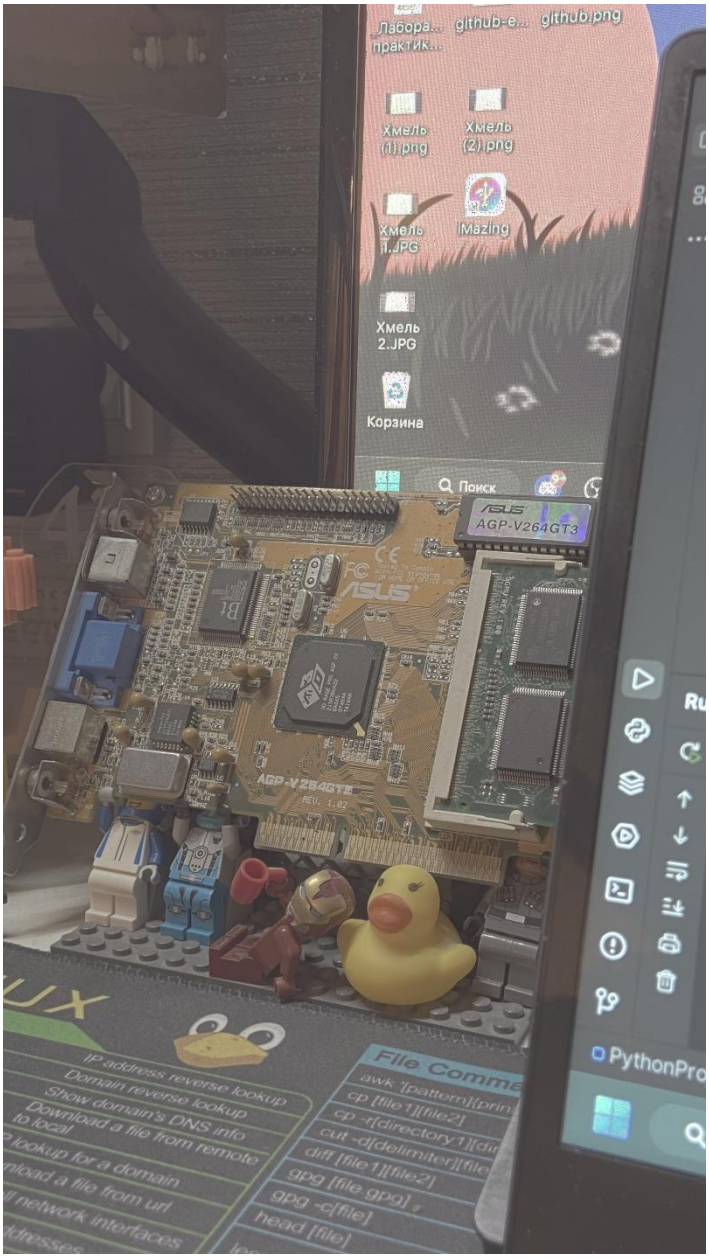

Рис. 5. Результат HDR + Mantiuk.

Рис. 6. Итоговый результат

Сравнение исходных LDR-кадров и результатов тонального отображения показывает следующее. В исходной серии ни один кадр не содержит одновременно детализированные тени и света: при короткой выдержке сохраняются яркие области, но теряются детали в тенях; при длинной выдержке тени становятся различимы, но часть уходит в пересвет.

После HDR-слияния и тонального отображения:

Reinhard (03_tonemap_reinhard.jpg, рис. 3) даёт наиболее естественный и “фотографический” результат: хорошо сохраняет общий баланс, без агрессивного поднятия теней.

Drago (04_tonemap_drago.jpg, рис. 4) сильнее проявляет детали в тенях за счёт логарифмического сжатия, но визуально может давать более “плоскую” картинку и повышенную заметность шума в темных областях.

Mantiuk (05_tonemap_mantiuk.jpg, рис. 5) заметно меняет глобальный контраст (через нормировку лог-яркости); при некоторых параметрах может переосветлять сцену или усиливать локальные перепады.

Дополнительно сформирован финальный вариант 06_FINAL_enhanced.jpg (рис. 6), в котором после Reinhard выполнена растяжка уровней и усиление насыщенности/контраста, что делает изображение визуально более “сочным” без существенной потери деталей.

4.3. Время выполнения и ресурсы

Реализация написана на Python с использованием NumPy для векторизации вычислений и решения задачи МНК при калибровке Дебевека. За счёт пирамидального MTB-выравнивания, обрезки общей области (crop) и векторных операций время обработки существенно снижено по сравнению с наивной реализацией на списках/циклах Python и остаётся комфортным для офлайн-обработки кадров высокого разрешения. Для одного набора из 5 кадров высокого разрешения основные временные затраты распределяются следующим образом:

- Выравнивание MTB: выполняется по яркости (grayscale) и использует пирамидальный подход (coarse-to-fine) с локальным поиском сдвига 3×3 на каждом уровне. Это резко снижает число проверяемых сдвигов по сравнению с перебором на полном разрешении.
- Обрезка общей области (crop): после выравнивания вычисляется пересечение валидных областей всех кадров и выполняется обрезка. Это

уменьшает размеры данных для последующей калибровки/слияния/тонмаппинга и ускоряет конвейер, одновременно снижая влияние “чёрных полей” на итоговый результат.

- Калибровка Debevec: выполняется по случайной выборке из samples точек (порядка нескольких сотен) и решается как задача наименьших квадратов (np.linalg.lstsq), поэтому не требует обработки всех пикселей при построении системы.
- HDR-слияние: реализовано векторно в NumPy: для каждого канала накапливаются числитель/знаменатель по всем экспозициям, после чего восстанавливается радианс через экспоненту.
- Тональное отображение и постобработка: выполняются векторно; дополнительно финальный кадр улучшается растяжкой уровней и корректировкой насыщенности/контраста средствами Pillow.

В результате суммарное время работы в типичном окружении составляет несколько секунд (зависит от разрешения кадров, параметров mtb_min_size, samples, а также от производительности процессора), что значительно быстрее реализации без NumPy.

В целом, сравнительный анализ показывает, что реализованный HDR-конвейер корректно объединяет информацию из нескольких LDR-кадров и позволяет восстановить детали, отсутствующие в любом отдельном исходном кадре. Выбор оператора тонального отображения критически влияет на вид результата, и каждый из реализованных операторов (Reinhard, Drago, Mantiuk) имеет свои уникальные отличия по контрасту, шуму и визуальной естественности. Несмотря на отсутствие низкоуровневых оптимизаций, время работы алгоритма приемлемо для офлайн-обработки изображений высокого разрешения.

Заключение

В ходе выполнения лабораторной работы был разработан и реализован полный программный конвейер построения HDR-изображения из серии LDR-кадров с разной экспозицией. Алгоритм включает в себя все основные этапы классического HDR-подхода:

1) Предобработка и выравнивание кадров.

- Реализован метод выравнивания изображений на основе MTB (Median Threshold Bitmap). Для каждого кадра формируется бинарная карта яркости, и по минимуму суммы модулей разности (SAD) подбирается глобальный сдвиг относительно опорного кадра. Это позволяет компенсировать небольшие смещения камеры по контурам при HDR-слиянии.

2) Калибровка и построение HDR:

- На основе метода Дебевека сформирована и решена переопределённая система линейных уравнений, связывающая истинную яркость сцены, выдержку и отклик камеры:
    - Построена функция отклика $g(z)$ для диапазона уровней яркости 0–255.
    - На основе найденной функции отклика и известных выдержек восстановлена карта радианса сцены (HDR-изображение) по всем пикселям и цветовым каналам.

3) Тональное отображение

- Реализованы три оператора тонального отображения:
    - Reinhard – глобальный оператор, дающий наиболее естественный и сбалансированный результат;
    - Drago – логарифмический оператор, агрессивно подтягивающий тени и сильно сжимающий динамический диапазон, что сопровождается сероватым видом и усилением шума;

- Mantiuk – оператор на основе нормировки лог-яркости, при выбранных параметрах приводящий к сильному осветлению сцены и частичному выгоранию светлых областей.

Сравнение показало, что выбор тонмаппера существенно влияет на субъективное восприятие результата, хотя исходная HDR-карта при этом остаётся одной и той же.

4) Программная реализация

- Все этапы реализованы на языке Python без использования специализированных HDR-библиотек. Несмотря на отсутствие низкоуровневых оптимизаций, программа успешно обрабатывает кадры высокого разрешения и демонстрирует корректное восстановление деталей в тенях и светах.

Практическая ценность.

Результаты подтверждают, что из серии LDR-кадров удается восстановить информацию, отсутствующую на любом одном исходном снимке; глобальный оператор Reinhard дает наиболее правдоподобное изображение; операторы Drago и Mantiuk хорошо демонстрируют, как различные стратегии тонального отображения по-разному распределяют яркости, усиливают или ослабляют детали и шум.

Перспективы развития.

Выявленные в ходе работы нюансы и особенности алгоритмов, можно выделить направления дальнейшей работы:

- использование продвинутых алгоритмов выравнивания (feature-based, optical flow) для более сложных сцен;
- автоматический подбор параметров экспозиции и тонмаппинга на основе целевых метрик (SSIM, TMQI) или методов машинного обучения;

- включение в работу многопоточных вычислений, для ускорения и стабильности конвейера.

В целом, поставленная цель – реализация алгоритма HDR-формирования с нуля на основе метода Дебевека и нескольких операторов тонального отображения – достигнута, а полученные результаты демонстрируют как теоретическую корректность подхода, так и его практическую работоспособность.

Список используемых источников

1. Debevec P. E., Malik J. Recovering High Dynamic Range Radiance Maps from Photographs // Proceedings of SIGGRAPH 1997. — 1997. — С. 369–378.
2. Reinhard E., Devlin K. Dynamic Range Reduction Inspired by Photoreceptor Physiology // IEEE Transactions on Visualization and Computer Graphics. — 2005. — Т. 11, № 1. — С. 13–24.
3. Reinhard E., Ward G., Pattanaik S., Debevec P. High Dynamic Range Imaging: Acquisition, Display, and Image-Based Lighting. — 2-е изд. — Morgan Kaufmann, 2 gef. — 672 с.
4. Drago F., Myszkowski K., Annen T., Chiba N. Adaptive Logarithmic Mapping for Displaying High Contrast Scenes // Computer Graphics Forum. — 2003. — Т. 22, № 3. — С. 419–426.
5. Mantiuk R., Daly S., Kerofsky L. Display Adaptive Tone Mapping // ACM Transactions on Graphics (Proc. SIGGRAPH). — 2008. — Т. 27, № 3. — Статья 68.
6. Ward G. Fast, Robust Image Registration for Compositing High Dynamic Range Photographs // Journal of Graphics Tools. — 2003. — Т. 8, № 2. — С. 17–30. (Метод MTB-выравнивания.)
7. OpenCV Documentation [Электронный ресурс]. – Режим доступа: https://docs.opencv.org/4.x/ (дата обращения: 25.11.2025).

Приложение

```
import argparse
import math
import os
from dataclasses import dataclass
from typing import List, Tuple

import numpy as np
from PIL import Image, ImageEnhance

def read_image_rgb(path: str) -> np.ndarray:
    img = Image.open(path).convert("RGB")
    return np.array(img, dtype=np.uint8)

def save_image_rgb(path: str, rgb_u8: np.ndarray, quality: int = 95) -> None:
    os.makedirs(os.path.dirname(path), exist_ok=True)
    Image.fromarray(rgb_u8, mode="RGB").save(path, format="JPEG", quality=quality, optimize=True)

def shift_image_zero_pad(img: np.ndarray, dx: int, dy: int) -> np.ndarray:
    h, w = img.shape[:2]
    out = np.zeros_like(img)

    x_src0 = max(0, -dx)
    x_src1 = min(w, w - dx)
    y_src0 = max(0, -dy)
    y_src1 = min(h, h - dy)

    x_dst0 = max(0, dx)
    x_dst1 = min(w, w + dx)
    y_dst0 = max(0, dy)
    y_dst1 = min(h, h + dy)

    if x_src1 <= x_src0 or y_src1 <= y_src0:
        return out

    out[y_dst0:y_dst1, x_dst0:x_dst1, ...] = img[y_src0:y_src1, x_src0:x_src1, ...]
    return out

def compute_common_crop(shifts: List[Tuple[int, int]], w: int, h: int) -> Tuple[int, int, int, int]:
    dxs = [dx for dx, _ in shifts]
    dys = [dy for _, dy in shifts]

    left = max(0, max(dxs))
    right = min(w, min(w + dx for dx in dxs))
    top = max(0, max(dys))
    bottom = min(h, min(h + dy for dy in dys))

    left = int(left); right = int(right); top = int(top); bottom = int(bottom)
    if right - left < 10 or bottom - top < 10:
        return 0, 0, w, h
    return left, top, right, bottom

def rgb_to_gray_u8(rgb: np.ndarray) -> np.ndarray:
    r = rgb[..., 0].astype(np.float32)
    g = rgb[..., 1].astype(np.float32)
    b = rgb[..., 2].astype(np.float32)
    gray = 0.299 * r + 0.587 * g + 0.114 * b
    return np.clip(gray + 0.5, 0, 255).astype(np.uint8)
```

```
def resize_gray(gray_u8: np.ndarray, new_w: int) -> np.ndarray:
    h, w = gray_u8.shape
    if new_w >= w:
        return gray_u8
    new_h = max(1, int(round(h * (new_w / w))))
    im = Image.fromarray(gray_u8, mode="L")
    im = im.resize((new_w, new_h), resample=Image.BILINEAR)
    return np.array(im, dtype=np.uint8)

def build_pyramid(gray_u8: np.ndarray, min_size: int = 64) -> List[np.ndarray]:
    pyr = [gray_u8]
    while min(pyr[-1].shape) > min_size:
        h, w = pyr[-1].shape
        new_w = max(min_size, w // 2)
        pyr.append(resize_gray(pyr[-1], new_w))
    return pyr[::-1]  # coarse -> fine

def mtb_and_exclusion(gray_u8: np.ndarray, tolerance: int = 6) -> Tuple[np.ndarray, np.ndarray]:
    med = np.median(gray_u8)
    mtb = gray_u8 >= med
    excl = np.abs(gray_u8.astype(np.int16) - int(med)) > tolerance
    return mtb, excl

def mtb_error(mtb_ref: np.ndarray, excl_ref: np.ndarray,
              mtb_cur: np.ndarray, excl_cur: np.ndarray,
              dx: int, dy: int) -> int:

    mtb_cur_s = shift_image_zero_pad(mtb_cur.astype(np.uint8), dx, dy).astype(bool)
    excl_cur_s = shift_image_zero_pad(excl_cur.astype(np.uint8), dx, dy).astype(bool)

    mask = excl_ref & excl_cur_s
    diff = (mtb_ref ^ mtb_cur_s) & mask
    return int(diff.sum())


def align_mtb_translation(images_rgb: List[np.ndarray],
                          ref_index: int,
                          pyramid_min_size: int = 64,
                          tolerance: int = 6) -> Tuple[List[np.ndarray], List[Tuple[int, int]]]:

    grays = [rgb_to_gray_u8(im) for im in images_rgb]
    ref_gray = grays[ref_index]

    ref_pyr = build_pyramid(ref_gray, min_size=pyramid_min_size)
    pyrs = [build_pyramid(g, min_size=pyramid_min_size) for g in grays]

    shifts: List[Tuple[int, int]] = []
    aligned: List[np.ndarray] = []

    for i in range(len(images_rgb)):
        if i == ref_index:
            shifts.append((0, 0))
            aligned.append(images_rgb[i])
            continue

        dx, dy = 0, 0
        for lvl in range(len(ref_pyr)):
            ref_lvl = ref_pyr[lvl]
            cur_lvl = pyrs[i][lvl]
```

```
            mtb_ref, excl_ref = mtb_and_exclusion(ref_lvl, tolerance=tolerance)
            mtb_cur, excl_cur = mtb_and_exclusion(cur_lvl, tolerance=tolerance)

            if lvl > 0:
                dx *= 2
                dy *= 2

            best = (10**18, dx, dy)
            for ddx in (-1, 0, 1):
                for ddy in (-1, 0, 1):
                    cand_dx = dx + ddx
                    cand_dy = dy + ddy
                    err = mtb_error(mtb_ref, excl_ref, mtb_cur, excl_cur, cand_dx, cand_dy)
                    if err < best[0]:
                        best = (err, cand_dx, cand_dy)

            dx, dy = best[1], best[2]

        shifts.append((dx, dy))
        aligned.append(shift_image_zero_pad(images_rgb[i], dx, dy))

    return aligned, shifts

def weight_triangle(z_u8: np.ndarray) -> np.ndarray:
    z = z_u8.astype(np.int16)
    return np.minimum(z, 255 - z).astype(np.float32)

def pick_sample_points(mid_img_u8: np.ndarray,
                       n_samples: int = 300,
                       valid_range: Tuple[int, int] = (10, 245),
                       seed: int = 42) -> np.ndarray:
    rng = np.random.default_rng(seed)
    h, w = mid_img_u8.shape
    mask = (mid_img_u8 >= valid_range[0]) & (mid_img_u8 <= valid_range[1])
    ys, xs = np.where(mask)
    if len(ys) == 0:
        ys = rng.integers(0, h, size=n_samples)
        xs = rng.integers(0, w, size=n_samples)
        return np.stack([ys, xs], axis=1)

    idx = rng.choice(len(ys), size=min(n_samples, len(ys)), replace=False)
    return np.stack([ys[idx], xs[idx]], axis=1)

def levels_stretch_u8(rgb_u8: np.ndarray, p_black: float = 0.5, p_white: float = 99.5) -> np.ndarray:
    x = rgb_u8.astype(np.float32) / 255.0
    out = np.empty_like(x)

    for c in range(3):
        lo = np.percentile(x[..., c], p_black)
        hi = np.percentile(x[..., c], p_white)
        if hi <= lo + 1e-6:
            out[..., c] = x[..., c]
        else:
            out[..., c] = (x[..., c] - lo) / (hi - lo)

    out = np.clip(out, 0.0, 1.0)
    return (out * 255.0 + 0.5).astype(np.uint8)

def enhance_pil_pop(rgb_u8: np.ndarray,
```

```
                color: float = 1.9,
                contrast: float = 1.25,
                brightness: float = 1.05,
                sharpness: float = 1.2) -> np.ndarray:
    img = Image.fromarray(rgb_u8, mode="RGB")
    img = ImageEnhance.Color(img).enhance(color)
    img = ImageEnhance.Contrast(img).enhance(contrast)
    img = ImageEnhance.Brightness(img).enhance(brightness)
    img = ImageEnhance.Sharpness(img).enhance(sharpness)
    return np.array(img, dtype=np.uint8)

def solve_debevec_response(Z_samples: np.ndarray,
                           ln_t: np.ndarray,
                           lambda_reg: float = 100.0) -> np.ndarray:
    P, N = Z_samples.shape
    nZ = 256
    n_eq = P * N + 1 + (nZ - 2)
    n_unknown = nZ + P  # g(0..255) + lnE_i

    A = np.zeros((n_eq, n_unknown), dtype=np.float64)
    b = np.zeros((n_eq,), dtype=np.float64)

    k = 0
    for i in range(P):
        for j in range(N):
            z = int(Z_samples[i, j])
            w = float(min(z, 255 - z))
            if w <= 0.0:
                continue
            A[k, z] = w
            A[k, nZ + i] = w
            b[k] = w * ln_t[j]
            k += 1
    A[k, 128] = 1.0
    b[k] = 0.0
    k += 1

    for z in range(1, nZ - 1):
        w = lambda_reg * min(z, 255 - z)
        A[k, z - 1] = w
        A[k, z] = -2.0 * w
        A[k, z + 1] = w
        b[k] = 0.0
        k += 1

    A = A[:k, :]
    b = b[:k]

    x, *_ = np.linalg.lstsq(A, b, rcond=None)
    g = x[:nZ]
    return g


def merge_hdr_debevec(images_u8: np.ndarray,
                      times: np.ndarray,
                      g_channels: List[np.ndarray]) -> np.ndarray:
    N, H, W, _ = images_u8.shape
    ln_times = np.log(times.astype(np.float64) + 1e-12)  # (N,)

    hdr = np.zeros((H, W, 3), dtype=np.float32)
```

```
    for ch in range(3):
        g = g_channels[ch].astype(np.float64)
        num = np.zeros((H, W), dtype=np.float64)
        den = np.zeros((H, W), dtype=np.float64)

        for j in range(N):
            Z = images_u8[j, ..., ch]                # (H,W) uint8
            Wgt = weight_triangle(Z).astype(np.float64) # (H,W) float64
            Gj = g[Z]                                # (H,W) float64
            num += Wgt * (Gj - ln_times[j])
            den += Wgt

        lnE = num / (den + 1e-12)
        hdr[..., ch] = np.exp(lnE).astype(np.float32)

    return hdr

def to_uint8(rgb01: np.ndarray) -> np.ndarray:
    return np.clip(rgb01 * 255.0 + 0.5, 0, 255).astype(np.uint8)

def auto_expose_ldr(rgb01: np.ndarray, percentile: float = 99.0, target: float = 0.95) -> np.ndarray:
    rgb01 = np.clip(rgb01, 0.0, 1.0).astype(np.float32)
    L = 0.2126 * rgb01[..., 0] + 0.7152 * rgb01[..., 1] + 0.0722 * rgb01[..., 2]
    p = float(np.percentile(L, percentile))
    gain = target / (p + 1e-6)
    return np.clip(rgb01 * gain, 0.0, 1.0)

def tonemap_reinhard(hdr: np.ndarray, key: float = 0.36, gamma: float = 2.2) -> np.ndarray:
    hdr = np.maximum(hdr, 0.0).astype(np.float32)
    L = 0.2126 * hdr[..., 0] + 0.7152 * hdr[..., 1] + 0.0722 * hdr[..., 2]
    delta = 1e-6
    Lavg = np.exp(np.mean(np.log(delta + L)))
    Lm = (key / (Lavg + 1e-12)) * L
    Ld = Lm / (1.0 + Lm)
    scale = Ld / (L + 1e-12)
    out = hdr * scale[..., None]
    out = np.clip(out, 0.0, 1.0)
    out = auto_expose_ldr(out, percentile=99.0, target=0.95)
    out = out ** (1.0 / gamma)
    return to_uint8(out)

def tonemap_drago_simple(hdr: np.ndarray, bias: float = 0.85, gamma: float = 2.2) -> np.ndarray:
    hdr = np.maximum(hdr, 0.0).astype(np.float32)
    L = 0.2126 * hdr[..., 0] + 0.7152 * hdr[..., 1] + 0.0722 * hdr[..., 2]
    Lmax = float(np.max(L)) + 1e-12

    b = np.clip(bias, 1e-4, 0.9999)
    exponent = math.log(b) / math.log(0.5)

    Ld = np.log1p(L) / np.log1p(Lmax)
    Ld = np.power(np.clip(Ld, 0.0, 1.0), exponent)

    scale = Ld / (L + 1e-12)
    out = hdr * scale[..., None]
    out = np.clip(out, 0.0, 1.0)
    out = out ** (1.0 / gamma)
    return to_uint8(out)
```

```
def tonemap_mantiuk_simple(hdr: np.ndarray, scale: float = 0.7, saturation: float = 1.0, gamma: float = 2.2) -> np.ndar-
ray:
    hdr = np.maximum(hdr, 0.0).astype(np.float32)
    L = (hdr[..., 0] + hdr[..., 1] + hdr[..., 2]) / 3.0
    logL = np.log(L + 1e-6)
    mu = float(np.mean(logL))
    sigma = float(np.std(logL)) + 1e-6

    z = (logL - mu) / sigma
    z = z * float(scale)
    Ld = np.exp(z)

    Rn = hdr[..., 0] / (L + 1e-12)
    Gn = hdr[..., 1] / (L + 1e-12)
    Bn = hdr[..., 2] / (L + 1e-12)

    sat = float(saturation)
    Rn = np.power(np.clip(Rn, 0, 10), sat)
    Gn = np.power(np.clip(Gn, 0, 10), sat)
    Bn = np.power(np.clip(Bn, 0, 10), sat)

    out = np.stack([Ld * Rn, Ld * Gn, Ld * Bn], axis=-1)
    p = np.percentile(out, 99.5)
    out = out / (p + 1e-12)
    out = np.clip(out, 0.0, 1.0)
    out = out ** (1.0 / gamma)
    return to_uint8(out)

def enhance_simple(ldr_u8: np.ndarray, saturation: float = 1.25, contrast: float = 1.05, sharpen: float = 0.4) -> np.ndar-
ray:
    rgb = ldr_u8.astype(np.float32) / 255.0

    rgb = np.clip((rgb - 0.5) * contrast + 0.5, 0.0, 1.0)

    r, g, b = rgb[..., 0], rgb[..., 1], rgb[..., 2]
    cmax = np.max(rgb, axis=-1)
    cmin = np.min(rgb, axis=-1)
    delta = cmax - cmin
    s = np.where(cmax > 1e-12, delta / (cmax + 1e-12), 0.0)
    s = np.clip(s * saturation, 0.0, 1.0)
    gray = (r + g + b) / 3.0
    rgb = gray[..., None] + (rgb - gray[..., None]) * (s[..., None] / (np.where(delta > 1e-12, s, 1.0)[..., None] + 1e-12))

    rgb = np.clip(rgb, 0.0, 1.0)

    if sharpen > 1e-6:
        pad = np.pad(rgb, ((1, 1), (1, 1), (0, 0)), mode="edge")
        blur = (
            pad[0:-2, 0:-2] + pad[0:-2, 1:-1] + pad[0:-2, 2:] +
            pad[1:-1, 0:-2] + pad[1:-1, 1:-1] + pad[1:-1, 2:] +
            pad[2:, 0:-2] + pad[2:, 1:-1] + pad[2:, 2:]
        ) / 9.0
        rgb = np.clip(rgb + sharpen * (rgb - blur), 0.0, 1.0)

    return to_uint8(rgb)

@dataclass
class HDRParams:
    mtb_tolerance: int = 6
    mtb_min_size: int = 64
```

```
    samples: int = 350
    lambda_reg: float = 100.0

def run_pipeline(image_paths: List[str],
                 exposure_times: List[float],
                 out_dir: str,
                 params: HDRParams) -> None:
    if len(image_paths) != len(exposure_times):
        raise ValueError("Количество изображений должно совпадать с количеством выдержек")

    print(f"Загрузка {len(image_paths)} изображений...")
    images = [read_image_rgb(p) for p in image_paths]
    times = np.array(exposure_times, dtype=np.float32)

    shapes = [im.shape for im in images]
    if len(set(shapes)) != 1:
        raise ValueError(f"Все изображения должны быть одного размера. Сейчас: {set(shapes)}")

    h, w = images[0].shape[:2]
    ref_index = len(images) // 2
    print(f"Опорный кадр: #{ref_index+1} ({os.path.basename(image_paths[ref_index])})")

    print("Выравнивание MTB (сдвиг dx/dy)...")
    aligned, shifts = align_mtb_translation(
        images, ref_index=ref_index,
        pyramid_min_size=params.mtb_min_size,
        tolerance=params.mtb_tolerance
    )

    for i, (dx, dy) in enumerate(shifts):
        print(f"  кадр {i+1}: shift dx={dx}, dy={dy}")

    x0, y0, x1, y1 = compute_common_crop(shifts, w, h)
    aligned = [im[y0:y1, x0:x1, :] for im in aligned]
    images_stack = np.stack(aligned, axis=0)  # (N,H,W,3)
    Hc, Wc = images_stack.shape[1], images_stack.shape[2]
    print(f"После crop общий размер: {Wc}x{Hc}")

    os.makedirs(out_dir, exist_ok=True)
    for i, im in enumerate(images):
        save_image_rgb(os.path.join(out_dir, f"01_original_{i+1}.jpg"), im)
    for i, im in enumerate(aligned):
        save_image_rgb(os.path.join(out_dir, f"02_aligned_{i+1}.jpg"), im)

    print("Debevec: калибровка отклика g(z) по каналам...")
    ln_t = np.log(times.astype(np.float64) + 1e-12)
    mid = images_stack[ref_index]  # (H,W,3)

    g_channels = []
    for ch, name in enumerate(["R", "G", "B"]):
        print(f"  канал {name}: выборка {params.samples} точек...")
        pts = pick_sample_points(mid[..., ch], n_samples=params.samples, seed=42 + ch)
        Z_samples = np.stack([images_stack[:, y, x, ch] for (y, x) in pts], axis=0)  # (P,N)
        g = solve_debevec_response(Z_samples, ln_t, lambda_reg=params.lambda_reg)
        g_channels.append(g)
        print(f"    g({name}) готов")

    print("Слияние в HDR (радианс-карта)...")
    hdr = merge_hdr_debevec(images_stack, times, g_channels)
```

```
    print("Tone mapping: Reinhard / Drago / Mantiuk ...")
    reinh = tonemap_reinhard(hdr, key=0.18, gamma=2.2)
    drago = tonemap_drago_simple(hdr, bias=0.85, gamma=2.2)
    mant = tonemap_mantiuk_simple(hdr, scale=0.7, saturation=1.0, gamma=2.2)

    save_image_rgb(os.path.join(out_dir, "03_tonemap_reinhard.jpg"), reinh)
    save_image_rgb(os.path.join(out_dir, "04_tonemap_drago.jpg"), drago)
    save_image_rgb(os.path.join(out_dir, "05_tonemap_mantiuk.jpg"), mant)

    base = reinh
    base = levels_stretch_u8(base, p_black=0.5, p_white=99.5)
    final = enhance_pil_pop(base, color=2.0, contrast=1.28, brightness=1.06, sharpness=1.25)
    save_image_rgb(os.path.join(out_dir, "06_FINAL_enhanced.jpg"), final)

    print("\nГотово.")
    print(f"Результаты: {out_dir}")
    print("Главный файл: 06_FINAL_enhanced.jpg")

def parse_times(s: str) -> List[float]:
    parts = [p.strip() for p in s.split(",") if p.strip()]
    return [float(p) for p in parts]

def list_images(folder: str) -> List[str]:
    exts = (".jpg", ".jpeg", ".png", ".bmp", ".tif", ".tiff")
    files = [os.path.join(folder, f) for f in sorted(os.listdir(folder)) if f.lower().endswith(exts)]
    if not files:
        raise ValueError(f"В папке {folder} не найдено изображений")
    return files

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--input", type=str, default="images", help="Папка с входными кадрами")
    ap.add_argument(
        "--times",
        type=str,
        default="0.02,0.0333333333,0.04,0.05,0.0666666667",
        help="Выдержки в секундах через запятую (по умолчанию: 1/50,1/30,1/25,1/20,1/15)"
    )
    ap.add_argument("--output", type=str, default="hdr_results_no_cv2", help="Папка для результатов")
    ap.add_argument("--samples", type=int, default=350, help="Число точек для Debevec калибровки")
    ap.add_argument("--lambda_reg", type=float, default=100.0, help="Регуляризация гладкости g(z)")
    ap.add_argument("--mtb_tol", type=int, default=6, help="MTB exclusion tolerance")
    ap.add_argument("--mtb_min", type=int, default=64, help="Минимальный размер пирамиды MTB")
    args = ap.parse_args()

    image_paths = list_images(args.input)
    times = parse_times(args.times)

    if len(image_paths) < len(times):
        raise ValueError(f"В папке {args.input} изображений меньше, чем выдержек: {len(image_paths)} <
{len(times)}")
    image_paths = image_paths[:len(times)]

    params = HDRParams(
        mtb_tolerance=args.mtb_tol,
        mtb_min_size=args.mtb_min,
        samples=args.samples,
        lambda_reg=args.lambda_reg
    )
```

```
    run_pipeline(image_paths, times, args.output, params)

if __name__ == "__main__":
    main()
```